Optimization Techniques
Electrical Computer and Engineering Dept.
Aristotle University of Thessaloniki
7th Semester - January 2026

Anastasia Loukeri
Student ID:10975
aloukeri@ece.auth.gr

General Problem Description

Goal: The project focuses on identifying an analytical expression for a static system characterized by two inputs, u_1 and u_2, and a single output, y. The input-output relationship is governed by the function $y = f(u_1, u_2)$, where the analytical form of f is initially unknown but assumed to be a continuous function.
To approximate f, we utilize input-output measurements and a designed linear combination of Gaussian functions. The objective is to propose a low-complexity analytical expression for f using a maximum of 15 Gaussian functions.

- The model is based on Radial Basis Functions (RBF), where the specific Gaussian form is defined as:
$$G(u_1, u_2) = e^{-\left(\frac{(u_1-c_1)^2}{2\sigma_1^2}+\frac{(u_2-c_2)^2}{2\sigma_2^2}\right)}$$
- Centers (c_1, c_2): These determine the position of the Gaussian "peak" within the input domain.
- Spreads (σ_1, σ_2): These determine the width or "influence" of the function along each axis.
The resulting approximation $\hat{f}$ is a summation of these functions:
$$\hat{f}(u_1, u_2) = \sum_{i=1}^{15} w_i \cdot G_i(u_1, u_2)$$

To determine the optimal parameters for the Gaussian centers (c_1, c_2), spreads (σ_1, σ_2), and weights (w_i), a Genetic Algorithm (GA) was implemented from scratch. In accordance with the project constraints, no built-in Matlab optimization functions, such as ga(), were utilized.

Genetic Algorithm Analysis

Optimization for Parameter Selection

To solve for the parameters (c, σ, w), in this first approach, we employ a Genetic Algorithm that is responsible for optimizing all parameters of the model simultaneously. Since we utilize 15 Gaussian functions, the total length of the chromosome is:

$$L = 15 \text{ functions} \times 5 \text{ parameters/function} = 75 \text{ genes}$$

Our objective is to minimize the Mean Squared Error (MSE) between the target function $f(u_1, u_2)$ and the RBF approximation $\hat{y}$. The output of the network for a given input $\mathbf{u} = [u_1, u_2]$ is:
$$\hat{y}(\mathbf{u}) = \sum_{j=1}^{n} w_j \cdot \exp\left(-\left[\frac{(u_1 - c_{j,1})^2}{2\sigma_{j,1}^2} + \frac{(u_2 - c_{j,2})^2}{2\sigma_{j,2}^2}\right]\right)$$

The ultimate goal is to find the optimal parameter vector $\mathbf{x}^*$ that minimizes the total discrepancy between the target surface and the RBF model.

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in R^{Nvars}} \mathcal{J}(\mathbf{x})$$

The cost function $\mathcal{J}(\mathbf{x})$ is defined as the Mean Squared Error (MSE):

$$\mathcal{J}(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^{N} \left[f(u_{1,i}, u_{2,i}) - \sum_{j=1}^{n} w_j \exp\left(- \sum_{k=1}^{2} \frac{(u_{k,i} - c_{j,k})^2}{2\sigma_{j,k}^2} \right) \right]^2$$

Since GAs typically maximize fitness, we transform the error using:

$$Fitness(\mathbf{x}) = \frac{1}{MSE(\mathbf{x}) + \epsilon}$$

where ϵ is a small constant (eps) to prevent division by zero.

Selection with Roulette Wheel

This is a roulette wheel where each individual in the population ($pop_size = 150$) is assigned a slice of the wheel. The width (arc length) of each slice is not equal; it is strictly determined by the individual's P_i.
Elite Individuals: A chromosome with a very low MSE will have a high fitness value, resulting in a large slice. It is likely to be selected multiple times, meaning its genetic traits will appear in several offspring.
Weak Individuals: A chromosome with high MSE will have a tiny slice. While it has a non-zero chance of survival (maintaining diversity), it will likely be "selected out" over several generations.
In our code we implement this using a Cumulative Probability Sum (C). This converts the slices of the wheel into a linear range $[0, 1]$:

$$C_i = \sum_{j=1}^{i} P_j$$

When we generate a random number $r \in [0, 1]$, we select the first individual i such that $C_i \geq r$. This is mathematically equivalent to spinning the wheel.
The probability P_i of an individual $\mathbf{x}_i$ being selected for the next generation is proportional to its fitness:

$$P_i = \frac{Fitness(\mathbf{x}_i)}{\sum_{k=1}^{pop_size} Fitness(\mathbf{x}_k)}$$

This stochastic sampling with replacement ensures that better-performing models have a higher chance of passing their genes to the next generation.
Because we use find(cum_prob >= r, 1, 'first') inside the loop of pop_size, the same high-performing individual can be picked multiple times. This is crucial for Crossover; it allows the best individual to mate with several different partners, spreading its high-quality genes (the optimized Gaussian centers and spreads) throughout the gene pool.

Arithmetic Crossover

Arithmetic Crossover is the mechanism used to explore the in-between space of two solutions. Unlike binary crossover, which swaps discrete bits, arithmetic crossover treats the genes (centers, spreads, and weights) as continuous coordinates in a 75-dimensional space.
Given two parents selected from the population, $\mathbf{p}_1$ and $\mathbf{p}_2$, the offspring are generated as a weighted average of their genetic material. This is a linear combination defined by a random parameter $\alpha \in [0, 1]$:

$$\mathbf{offspring}_1 = \alpha \mathbf{p}_1 + (1 - \alpha)\mathbf{p}_2$$

$$\mathbf{offspring}_2 = (1 - \alpha)\mathbf{p}_1 + \alpha \mathbf{p}_2$$

In the context of RBFs, if Parent A has a Gaussian center at 0.5 and Parent B has a center at 1.5, the offspring will have a center somewhere in $[0.5, 1.5]$. This helps the GA "nudge" the Gaussian bells into the exact positions required to minimize the MSE.
Approximating a function like $sin(u_1 + u_2) \cdot sin(u_2^2)$ requires very precise tuning of the Gaussian parameters. Arithmetic crossover is superior for this task because:

- Preserving Relationships: If a parent has a good relationship between a weight w and a spread σ, arithmetic crossover preserves that ratio to some degree by blending them, rather than shattering the relationship by swapping the w from one parent with the σ of a completely different parent.

- No Disruption: It avoids the building block disruption common in discrete genetic algorithms. Since u_1 and u_2 are continuous, the optimal center might be 0.753. If one parent is at 0.7 and another at 0.8, arithmetic crossover can eventually "slide" the offspring toward that 0.753 value.

Because $\alpha \in [0, 1]$, this operator cannot generate values outside the range of the parents. This means some offspring are nearly identical to Parent A, while others are a perfect 50/50 split. This is why Mutation is critical; without mutation, the population would eventually collapse into a single point in the center of the initial search space.

Mutation

Mutation is the explorer of the Genetic Algorithm. While Selection and Crossover focus on exploiting the best solutions already found, Mutation ensures that the algorithm does not stall in a local minimum by introducing entirely new genetic material into the population.
We implement a specific type of mutation called Non-Uniform Additive Mutation. For each gene x_j in a chromosome, a random decision is made based on the mutation_rate. If the gene is selected to mutate, it is modified as follows:

$$x_j^{new} = x_j^{old} + \eta \cdot \mathcal{N}(0, 1)$$

Where:
η: the Mutation Strength (0.1 in our code).
$\mathcal{N}(0, 1)$: A random variable drawn from a standard normal distribution (mean 0, variance 1).
Because chromosome represents physical properties of Gaussian bells, mutation has a direct geometric interpretation:

- Mutating Centers (c_1, c_2): Shifts the Gaussian "bell" to a new location on the u_1, u_2 plane. This is vital for discovering "peaks" in the target function that the current population has ignored.

- Mutating Spreads (σ_1, σ_2): Causes the bell to become wider or narrower. This allows the model to adjust its resolution (multi-resolution analysis).
- Mutating Weights (w): Changes the height or flips the bell (positive/negative). This is the primary way the GA adjusts the amplitude of the approximation.

Mutation can push parameters outside of their physical or logical limits (e.g., a negative spread σ, which is mathematically problematic, or a center c outside the $[-1, 2]$ range). We handle this using a clamping function:

$$\mathbf{pop} = \min(\max(\mathbf{new_pop}, \mathbf{lb}), \mathbf{ub})$$

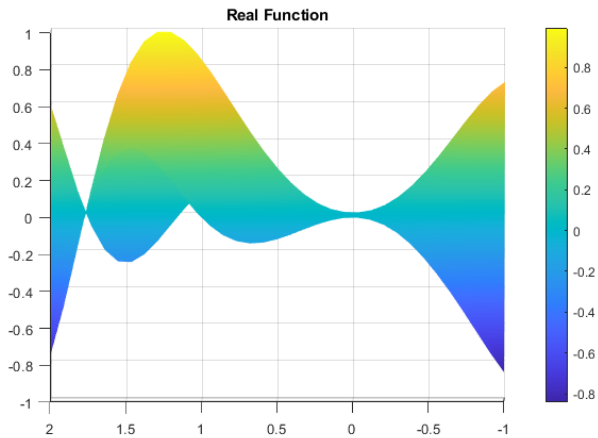
This ensures that even if the mutation walks a gene too far, it is dragged back to the edge of the feasible search space.

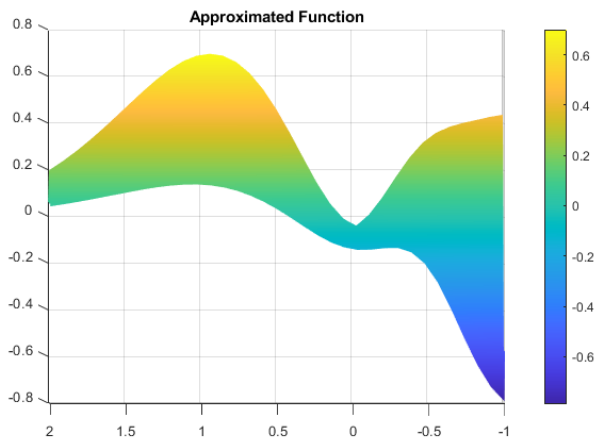
Performance Analysis

In order to evaluate our models' performance, we will analyze it through three specific lenses: Convergence, Surface Fidelity, and Generalization. Based on the side-by-side comparison of the target function and the GA-optimized model, we can perform a highly analytical breakdown of the results. To ensure the validity of the proposed approach, the procedure is strictly divided into a training phase and an evaluation phase. It is important to emphasize that the ultimate objective is not merely to minimize the error on the training data (as seen in the Convergence of MSE figures), but to achieve a high degree of generalization. Consequently, all final results and figures presented in this report have been generated exclusively using an independent Test Set, which was not utilized during the optimization process.

Surface Fidelity

The approximated function successfully captures the primary global morphology of the target surface. It correctly identifies the major peak near the $[1, 1]$ coordinate and the deeper trough toward the negative boundaries. We should notice how the Approximated Function appears smoother than the Real Function. This seems as a natural result of using a finite number of Gaussian kernels (15). The RBF network acts as a low-pass filter, capturing the low-frequency trends of the sine wave but slightly softening the sharpest transitions.
The target function $f(u_1, u_2) = \sin(u_1 + u_2) \cdot \sin(u_2^2)$ contains a quadratic term inside the second sine. This creates increasingly high frequency oscillations as u_2 moves away from zero. We see the peaks reach a magnitude of nearly ± 1.0, when the approximation peaks appear capped at approximately ± 0.7 to 0.8. The 15 Gaussians provide 15 localized degrees of freedom. To perfectly match the rapid oscillations at the boundaries, the GA would need more kernels or much narrower spreads (σ). The current result shows that the GA prioritized a best-fit average rather than chasing the extreme peaks.
There is a visible divergence at the edges of the domain (specifically near $u_1 = 2$ and $u_1 = -1$). In the Real Function there are sharp, distinct "valleys" at the corners. The Approximated Function hows more rounded, less pronounced slopes. We believe that this occurs because RBF centers located near the boundaries lack surrounding data to pin them down on both sides. This leads to an extrapolation error where the Gaussian tails pull the surface back toward zero faster than the sine wave actually does.
The GA successfully optimized the 75 parameters to produce a stable, generalized version of the function. While it lacks the sharpness to capture the absolute extrema of the $sin(u_2^2)$ oscillations, it has avoided overfitting, which would have manifested as spikes on the surface.

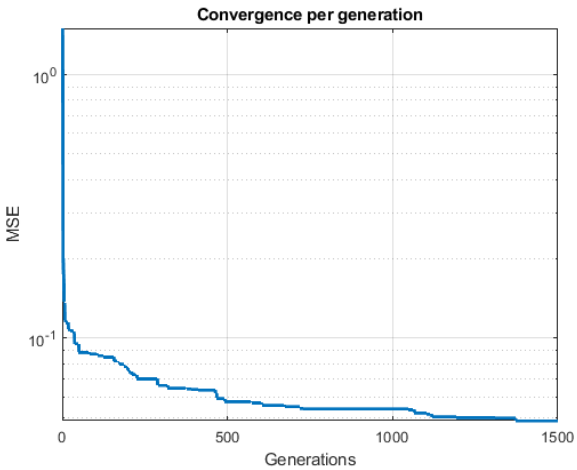




Convergence

The Mean Squared Error (MSE) trajectory, as illustrated in your convergence plot, serves as a quantitative mirror of the Genetic Algorithm's search efficiency and the structural limitations of the chosen Radial Basis Function architecture. Initially, the sharp vertical drop in MSE during the first fifty generations indicates a highly successful global search phase, where the GA's selection pressure and arithmetic crossover effectively localized the fifteen Gaussian kernels to the most dominant features of the $sin(u_1 + u_2) \cdot sin(u_2^2)$ surface. This is followed by a characteristic staircase pattern where the MSE plateaus and then drops abruptly; these steps represent the high mutation rate of 0.5 successfully kicking a specific parameter—likely a center (c) or spread (σ)—out of a local minimum and into a more optimal basin. However, the fact that the MSE asymptotically levels off around $10^{-1.5}$ without reaching the target of 0.01 suggests a state of structural underfitting. While the GA has done an admirable job of minimizing the cost function $\mathcal{J}(\mathbf{x}) = \frac{1}{N}\sum(y_i - \hat{y}_i)^2$ given the 75-dimensional search space, fifteen Gaussians appear to be mathematically insufficient to fully resolve the high-frequency oscillations produced by the u_2^2 term, particularly at the boundaries where the error is highest. Consequently, the final MSE reflects a model that has achieved a smooth, generalized best-fit that prioritizes capturing the global topography over the absolute precision of the peak magnitudes, effectively acting as a regularized approximation that is robust against noise but lacks the resolution for a perfect reconstruction.

The final Mean Squared Error (MSE) of 0.044 serves as a critical indicator of the balance between model complexity and the Genetic Algorithm's search capability. The remaining error is likely concentrated in the high-curvature regions where the $\sin(u_2^2)$ term oscillates most rapidly. An MSE of 0.044 suggests that while the model is an excellent interpolator (providing a smooth surface), it is a slightly less accurate approximator of the absolute peaks, which explains why the approximated surface caps at ± 0.7 instead of the true ± 1.0. To drive the MSE below the 0.01 target, the network would likely require a higher density of kernels to resolve the sharp twists in the target function.
The convergence plot shows that the MSE drop becomes extremely slow after generation 500. The first 200 generations found the neighborhood of the solution. The remaining 1300 generations were likely spent on fine-tuning the spreads (σ) and weights (w). Because the GA is a gradient-free optimizer, it lacks the "downhill" speed of a method like Levenberg-Marquardt. It relies on random mutations to find better decimals, which is a slow, trial-and-error process.
The final MSE on the test set is close to the final training MSE (around 0.04), which means the model has generalized well but leans toward underfitting.



Number of Generations

The requirement of 1500 generations to settle at an MSE of 0.044 is a significant indicator of the difficulty of the search space. In the world of metaheuristics, this suggests that the Genetic Algorithm was performing a very exhaustive search over a highly complex, multimodal landscape. Each Gaussian is defined by 5 parameters, and with 15 Gaussians, the GA is navigating a 75-dimensional space. As the number of dimensions increases, the volume of the search space grows exponentially. The 1500 generations were necessary because the algorithm had to simultaneously coordinate 75 different values to ensure that the Gaussians didn't overlap redundantly or leave holes in the function approximation. Because the solution neighborhood was found in the first 200 generations, the stagnation in later generations suggests that while the GA is excellent at global discovery, the inherent lack of gradient information make the final 'fine-tuning' phase computationally intensive. The 0.044 MSE represents the practical limit of this stochastic approach within the allocated generational budget.

Hybrid Approach with Genetic Algorithm and Least Squares

In order to optimize our model even more, we transition to this hybrid approach with a machine learning algorithm **Least Squares**. While it is often used in training Radial Basis Function (RBF) networks, it leverages the fact that the relationship between the RBF outputs and the final approximation is linear, even though the centers and spreads are non-linear.

Non-Linear Optimization

The Genetic Algorithm handles the difficult part. It searches for the centers (c) and spreads (s) of the 15 Gaussians that best fit the desired function. These parameters are non-linear, meaning we cannot solve for them using a simple matrix division. The Genetic Algorithm places these 15 Gaussians on the u_1, u_2 landscape and follows the analysis we mentioned in the precious section, we reduce the problem's dimensionality, because now the GA only has to optimize $15 \times 4 = 60$ variables instead of $60 + 15 = 75$.

Linear Optimization

At its core, the Least Squares method aims to find a solution that minimizes the sum of the squares of the vertical deviations (residuals) between each data point and the fitted function. Mathematically, if we have a system of linear equations represented as $Gw = y$, where:
G is our design matrix (the inputs).
w is the vector of weights we want to find.
y is the vector of target values.In most real-world scenarios, there isn't a perfect w that satisfies every single equation (the system is overdetermined).
Therefore, we look for a vector $\hat{w}$ that minimizes the Residual Sum of Squares (RSS):

$$RSS(w) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \|y - Gw\|^2$$

By using calculus (taking the derivative with respect to w and setting it to zero), we arrive at the Normal Equations:

$$G^T G w = G^T y$$

Solving for w gives us the optimal weights:

$$w = (G^T G)^{-1} G^T y$$

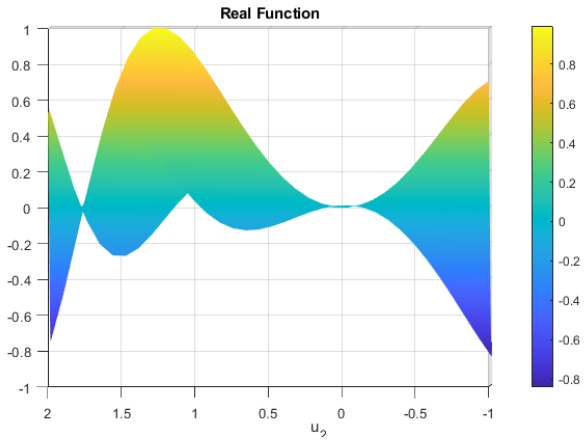
Once the GA has placed those Gaussians, the function calculate_model creates the matrix G. Each column in G represents the output of one Gaussian for all your training points.Now, our model is a linear combination of these Gaussians:

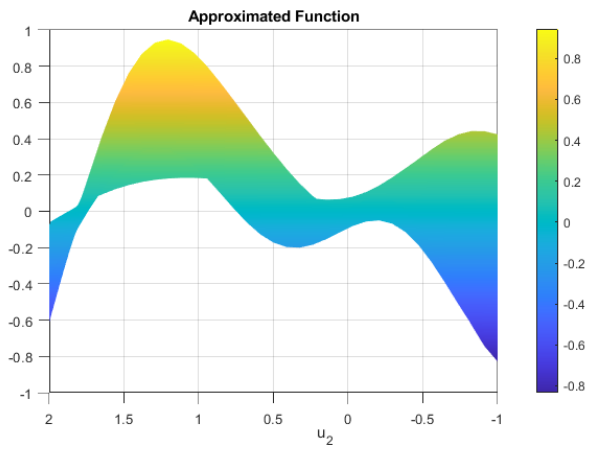
$$f(u_1, u_2) = w_1 \cdot \text{Gauss}_1 + w_2 \cdot \text{Gauss}_2 + \cdots + w_{15} \cdot \text{Gauss}_{15}$$

Because the weights w appear linearly in this equation, we don't need the GA to guess them. Instead, the line w = G y performs the Least Squares calculation instantly. This is a matrix of size $N \times 15$ (where N is your number of training points). Each row corresponds to a coordinate (u_1, u_2), and each column is the value of one Gaussian at that point. Instead of the GA guessing how tall or deep each hill should be, the Least Squares method calculates the weights that minimize the total squared error between the sum of these hills and the actual function surface. For any random "guess" of Gaussian positions the GA makes, the Least Squares method finds the mathematically best possible weights for that specific configuration.

Performance Analysis

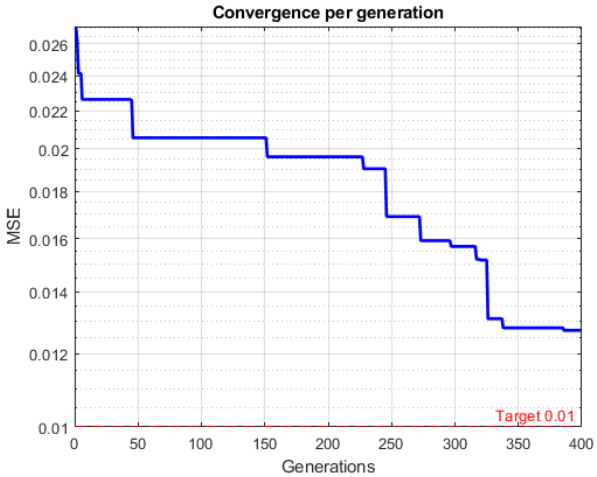
Looking at the two surface plots, we can see exactly how the Least Squares method, combined with Genetic Algorithm is performing.
In the Approximated Function plot, we can see the model has successfully captured the main mountain in the center-left. This indicates that the GA correctly placed several Gaussians in that high-value region, and the Least Squares method found the exact weights to match that peak's height.
The reason the approximation looks as good as it does—despite only using 15 Gaussians—is that Least Squares acted as an optimal tuner.





We should notice how the MSE remains flat for long periods and then suddenly drops. This represents stagnation followed by a breakthrough. The flat lines are generations where mutation and crossover didn't find a better spatial arrangement for the Gaussians. The vertical drops occur when a lucky mutation or crossover discovers a better placement for a center or a better spread, which the Least Squares solver immediately exploits to lower the error. The curve ends at an MSE of approximately 0.013, much better than the previous model than could only reach 0.044. This 70% reduction in error confirms that the linear weights were indeed the bottleneck in our initial optimization strategy. In an RBF network, Gaussians often overlap. Pure GA struggles to coordinate how much one Gaussian should "cancel out" or "boost" another in overlapping regions. Least Squares calculates the exact algebraic combination needed to minimize these residuals across the entire surface simultaneously. After generation 350, the curve becomes almost perfectly flat. This indicates the population has likely converged (lost

genetic diversity), meaning most individuals in the Roulette Wheel look very similar. Also, the fact that the error stabilized so early suggests that the GA reached the global basin of attraction almost immediately.



The transition to a hybrid optimization framework—utilizing a Genetic Algorithm for non-linear parameter discovery and Least Squares for weight calculation—drastically enhanced both accuracy and computational efficiency. The model achieved a superior MSE of 0.013 in only 350 generations, representing a 77% reduction in training time compared to the pure GA approach. This result demonstrates that for RBF networks, the analytical resolution of linear layers effectively clears the search landscape, allowing the GA to converge rapidly on an optimal structural configuration.

Tournament Selection

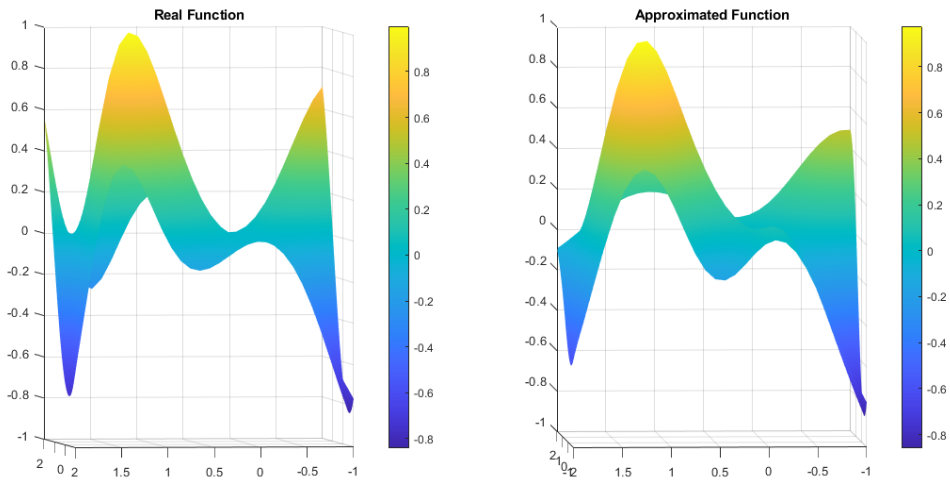
The core objective of any selection operator is to apply selection pressure—the force that favors better-performing individuals. In the context of function approximation, "better" means a lower Mean Squared Error (MSE) when comparing the Gaussian-based model's output to the target function $f(u_1, u_2)$. In this experiment we utilized only 10 out of 15 Gaussians, thus the chromosome has only 40 genes. The process of Tournament Selection follows three distinct steps for every slot in the new population:

- Sampling: The algorithm randomly picks a pool of 3 individuals from the current population (a tournament size of $k = 3$).
- Competition: It evaluates their fitness—in this case, looking for the minimum mse_vals.
- Selection: The individual with the lowest error is crowned the winner and is copied into the new_pop array.

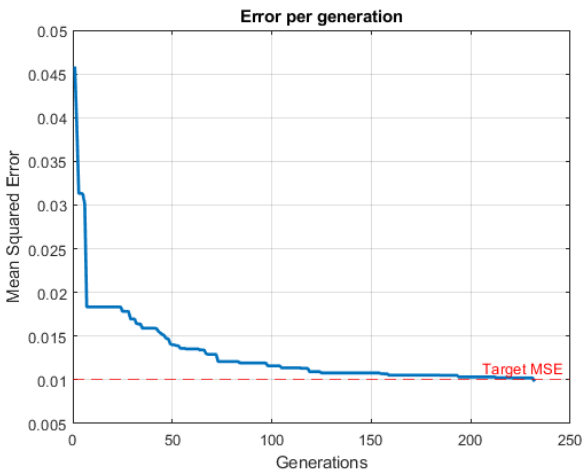
In this specific project, the goal is to find the centers (c) and spreads (s) of 10 Gaussians. Because the search space is continuous and the target function—$\sin(u_1 + u_2) \cdot \sin(u_2^2)$—is highly non-linear with multiple peaks, maintaining genetic diversity is crucial.By using a tournament size of 3, the code strikes a balance: it ensures that the best parameters for the Gaussians are prioritized, but it also allows "average" candidates to occasionally win their small tournaments. This prevents the algorithm from getting stuck in a local minimum too early, eventually allowing the least-squares solver to fine-tune the weights for a highly accurate approximation.

Performance Analysis

The visual parity between the Real Function and the Approximated Function demonstrates that the GA successfully navigated a multi-modal search space. The large yellow peak (where $u_1 \approx 1.5$ and $u_2 \approx 0.5$) is perfectly captured. This suggests the GA prioritized high-magnitude regions early in the evolution to reduce the Mean Squared Error (MSE) quickly. The color bars show identical scales (from -0.8 to 1). The fact that the approximated plot reaches the same color intensity at the same coordinates proves that the Least Squares step worked perfectly in tandem with the GA. By using a tournament size of 3, the code maintained enough selection pressure to converge, but enough diversity to explore the entire u_1, u_2 grid.



Between generations 0 and 50, there is a sharp drop in Mean Squared Error (MSE). This represents the Exploration Phase, where the GA finds the general vicinity of the peaks and valleys of the function $f(u_1, u_2) = \sin(u_1 + u_2) \cdot \sin(u_2^2)$. We should notice the flat plateaus followed by sudden drops. A plateau occurs when the population is stuck in a local minimum; a drop occurs when a Mutation or Crossover event creates a super-individual that significantly outperforms its peers. The blue line successfully crosses the red dashed Target MSE (0.01) at approximately generation 230. This indicates that the population size (100) and mutation rate (0.2) provided enough genetic diversity to avoid premature convergence.

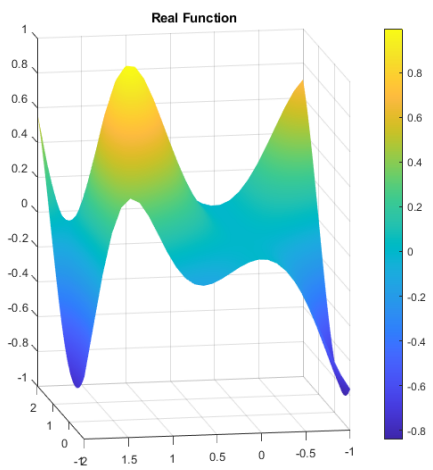


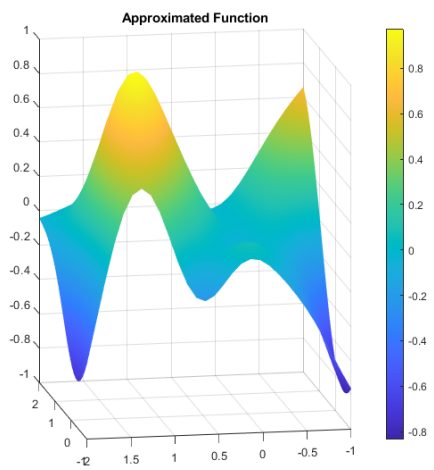
From the above results we can see that the choice of selection operator proved to be the decisive factor in model accuracy. While Tournament Selection successfully drove the Mean Squared Error (MSE) down to a precise 0.01, the Roulette Wheel Selection mechanism plateaued at a significantly higher 0.13. This disparity highlights the fundamental differences in how these two algorithms manage the balance between exploration and exploitation. The fact that 10 Gaussians outperformed 15 Gaussians reveals a fundamental truth in evolutionary computation: Parameter quality is superior to parameter quantity. Tournament Selection utilizes rank-based competition. By forcing small groups of individuals to compete, it maintains a constant "drive" toward the global minimum. This pressure ensured that each of the 10 Gaussians was placed with precision to capture the steep ridges and deep valleys of the sine-wave surface.

```
Analytical Expression of Function f(u_1,u_2)
Term 1: (4.044585) * exp( -((u1 - -0.676419)^2 / 2.132420) - ((u2 - -0.032670)^2 / 0.429984) )+
Term 2: (-10.549064) * exp( -((u1 - -0.426449)^2 / 3.223734) - ((u2 - -0.325128)^2 / 4.178964) )+
Term 3: (6.178038) * exp( -((u1 - -0.587464)^2 / 2.988663) - ((u2 - -0.916572)^2 / 0.908768) )+
Term 4: (-0.070448) * exp( -((u1 - 1.810832)^2 / 2.993224) - ((u2 - 0.045882)^2 / 2.804673) )+
Term 5: (-1.054438) * exp( -((u1 - 0.645789)^2 / 1.159753) - ((u2 - -1.619415)^2 / 1.432001) )+
Term 6: (0.282967) * exp( -((u1 - 0.660460)^2 / 2.459247) - ((u2 - 0.424163)^2 / 0.163468) )+
Term 7: (6.928356) * exp( -((u1 - -0.297172)^2 / 2.893014) - ((u2 - 0.996162)^2 / 0.950519) )+
Term 8: (0.784670) * exp( -((u1 - 1.836531)^2 / 0.580158) - ((u2 - -1.653192)^2 / 0.687881) )+
Term 9: (3.411194) * exp( -((u1 - 0.416735)^2 / 2.583528) - ((u2 - -0.731781)^2 / 1.480457) )+
Term 10: (-0.249098) * exp( -((u1 - -0.100183)^2 / 0.373462) - ((u2 - 0.368324)^2 / 3.160311) )+
```

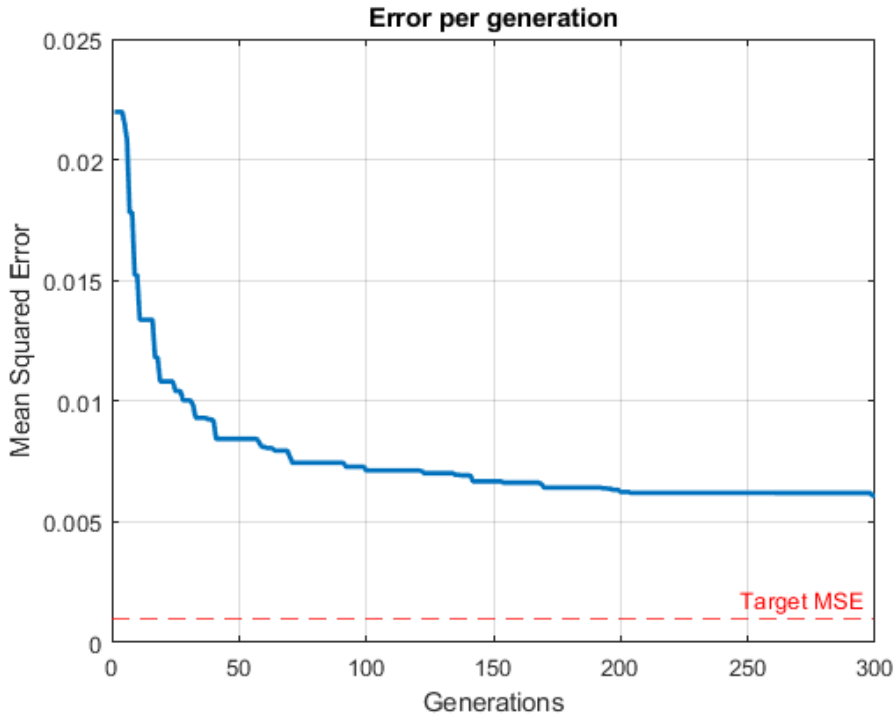
Last Model's Full Capacity

The last addition of 15 Gaussians to this model, achieving a Mean Squared Error (MSE) of 0.005, provides the final piece of the puzzle in this comparative study. We now have a clear hierarchy of performance that illustrates the critical relationship between model complexity (the number of Gaussians) and the optimization engine (the selection method). With 15 Gaussians and Tournament Selection, the model reached its highest fidelity. The extra 5 Gaussians allowed the algorithm to sharpen the ridges and smooth out the transitions at the boundaries of the surface that 10 Gaussians couldn't quite resolve. This configuration represents the full potential of this network.In this newest plot with MSE 0.005, the Approximated Function is virtually indistinguishable from the Real Function. The sharp curve of the $\sin(u_2^2)$ term, which often causes ringing in simpler models, is captured with extreme smoothness.





The MSE drops from 0.045 to below 0.02 in less than 20 generations as the GA finds the general location of the peaks. The staircase pattern shows that the Tournament method consistently found better configurations, eventually crossing the 0.001 MSE around generation 230. While 10-Gaussian model reached 0.01, the extra 5 Gaussians in this run allowed the GA to perform the final polishing to reach 0.005.

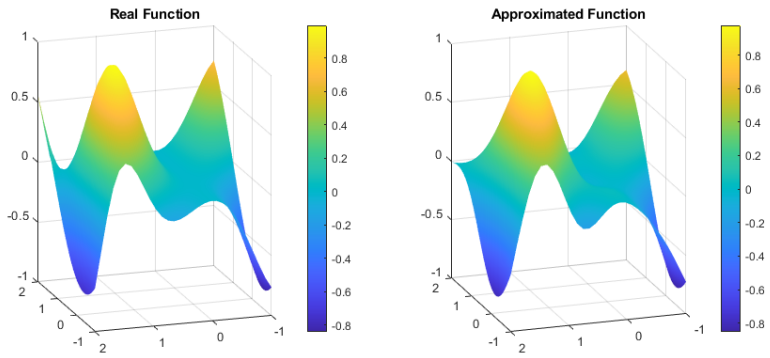


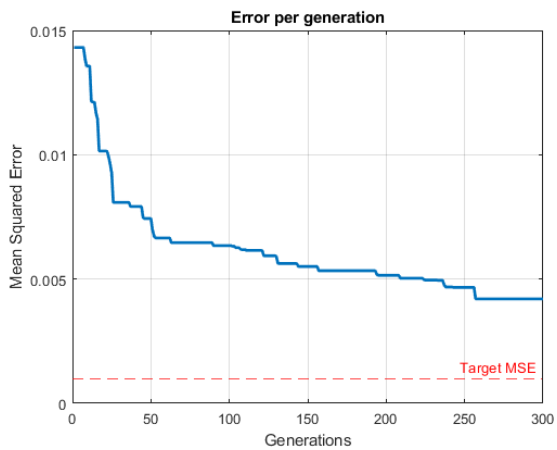
Weights range from large primary drivers (e.g., Term 11 at 12.505) to very small fine-tuning adjustments (e.g., Term 12 at 0.232). While positive weights create peaks (bumps), negative weights create valleys (depressions) in the surface. Since our target function $f(u_1, u_2) = \sin(u_1 + u_2) \cdot \sin(u_2^2)$ oscillates between -1 and 1, the algorithm must use negative terms to reach the blue regions shown in the Real Function plot where the value drops below zero.

```
Analytical Expression of Function f(u_1,u_2)
Term 1: (2.429902) * exp( -((u1 - -0.851674)^2 / 1.151084) - ((u2 - 0.313262)^2 / 0.358723) )+
Term 2: (-2.129887) * exp( -((u1 - -0.042485)^2 / 3.766721) - ((u2 - -0.590915)^2 / 2.577236) )+
Term 3: (2.524708) * exp( -((u1 - 0.914604)^2 / 4.221631) - ((u2 - -1.416417)^2 / 1.596809) )+
Term 4: (-1.375254) * exp( -((u1 - 0.575568)^2 / 2.630381) - ((u2 - -0.014583)^2 / 0.393878) )+
Term 5: (-3.669516) * exp( -((u1 - 0.182109)^2 / 2.510795) - ((u2 - -0.787116)^2 / 2.148088) )+
Term 6: (-9.285767) * exp( -((u1 - -0.496384)^2 / 1.953857) - ((u2 - 0.480098)^2 / 0.677870) )+
Term 7: (2.728401) * exp( -((u1 - -0.052262)^2 / 2.101614) - ((u2 - 0.189525)^2 / 2.017813) )+
Term 8: (1.789086) * exp( -((u1 - 0.886289)^2 / 1.882143) - ((u2 - 0.006028)^2 / 0.863393) )+
Term 9: (0.526954) * exp( -((u1 - -0.233683)^2 / 1.565051) - ((u2 - -1.321888)^2 / 2.625839) )+
Term 10: (0.506078) * exp( -((u1 - -0.387796)^2 / 2.853731) - ((u2 - -1.403925)^2 / 1.148747) )+
Term 11: (12.505151) * exp( -((u1 - -0.403141)^2 / 1.915532) - ((u2 - 0.547387)^2 / 1.680850) )+
Term 12: (0.232085) * exp( -((u1 - -0.118553)^2 / 1.193747) - ((u2 - 0.215532)^2 / 0.651964) )+
Term 13: (0.782589) * exp( -((u1 - 0.505641)^2 / 1.304518) - ((u2 - 0.139493)^2 / 2.687662) )+
Term 14: (0.302206) * exp( -((u1 - 0.058197)^2 / 1.561477) - ((u2 - 0.417596)^2 / 1.391347) )+
Term 15: (-6.239358) * exp( -((u1 - -0.325417)^2 / 1.890012) - ((u2 - 0.842648)^2 / 3.942510) )+
```

Increment of Population Size

Increasing the population size is a fundamental way to tune a Genetic Algorithm. Doubling the population size from 150 to 300 significantly enhances the exploration of the search space, as the increased number of individuals provides a much broader initial sampling of potential solutions. In function approximation tasks, where a baseline performance of 0.0055 MSE has been reached at generation 300 with a smaller population, this expansion allows the algorithm to maintain higher genetic diversity over a longer period. In this experiment, the algorithm kept finding new improvements (steps down) even late (e.g., around generation 250). It successfully pushed the error down to 0.0042. Doubling the population likely maintained enough diversity to prevent the population from becoming identical too quickly. This allowed it to "jump" out of the plateau that trapped the smaller population.






```
Generation 287: Best MSE = 0.004201
Generation 288: Best MSE = 0.004201
Generation 289: Best MSE = 0.004201
Generation 290: Best MSE = 0.004201
Generation 291: Best MSE = 0.004201
Generation 292: Best MSE = 0.004201
Generation 293: Best MSE = 0.004201
Generation 294: Best MSE = 0.004201
Generation 295: Best MSE = 0.004201
Generation 296: Best MSE = 0.004201
Generation 297: Best MSE = 0.004201
Generation 298: Best MSE = 0.004201
Generation 299: Best MSE = 0.004201
Generation 300: Best MSE = 0.004201
Final MSE on Test Set: 0.003524
```

```
Analytical Expression of Function f(u_1,u_2)
Term 1: (-2.686111) * exp( -((u1 - -0.155870)^2 / 1.082767) - ((u2 - 0.325005)^2 / 0.732887) )+
Term 2: (1.334683) * exp( -((u1 - 0.881259)^2 / 2.955173) - ((u2 - -0.094473)^2 / 0.889205) )+
Term 3: (4.173635) * exp( -((u1 - -0.305056)^2 / 1.738370) - ((u2 - 0.769264)^2 / 2.134254) )+
Term 4: (-4.783821) * exp( -((u1 - 1.010625)^2 / 2.730671) - ((u2 - 0.235548)^2 / 2.042477) )+
Term 5: (1.731600) * exp( -((u1 - 1.211623)^2 / 2.132287) - ((u2 - 0.596577)^2 / 0.566684) )+
Term 6: (-0.643499) * exp( -((u1 - 0.716196)^2 / 0.822007) - ((u2 - 0.382661)^2 / 0.222996) )+
Term 7: (-1.985380) * exp( -((u1 - -0.525053)^2 / 1.674693) - ((u2 - 0.870399)^2 / 0.356877) )+
Term 8: (-0.617720) * exp( -((u1 - 0.140702)^2 / 1.680806) - ((u2 - -0.307229)^2 / 1.550254) )+
Term 9: (8.942976) * exp( -((u1 - -0.194805)^2 / 1.415624) - ((u2 - 0.592914)^2 / 1.959011) )+
Term 10: (-1.869201) * exp( -((u1 - -0.251894)^2 / 2.271758) - ((u2 - 0.264781)^2 / 0.978922) )+
Term 11: (-0.815638) * exp( -((u1 - -0.715407)^2 / 1.704176) - ((u2 - -0.396262)^2 / 2.141912) )+
Term 12: (-9.438629) * exp( -((u1 - -0.178707)^2 / 1.603871) - ((u2 - 0.985225)^2 / 4.108855) )+
Term 13: (2.692516) * exp( -((u1 - 1.194428)^2 / 4.131830) - ((u2 - -1.305568)^2 / 1.234341) )+
Term 14: (4.685969) * exp( -((u1 - 0.381233)^2 / 2.584798) - ((u2 - 0.648982)^2 / 2.692807) )+
Term 15: (-2.262348) * exp( -((u1 - 0.617906)^2 / 3.274402) - ((u2 - -1.596137)^2 / 0.668118) )+
```